

PHẦN 2: QUẢN LÝ TIẾN TRÌNH

1. Tiến Trình
2. Luồng(**thread**)
3. Điều phối CPU
4. Tài nguyên găng và **đồng bộ** tiến trình
5. Bể tắc và xử lý bể tắc

1. Tiến trình (process)

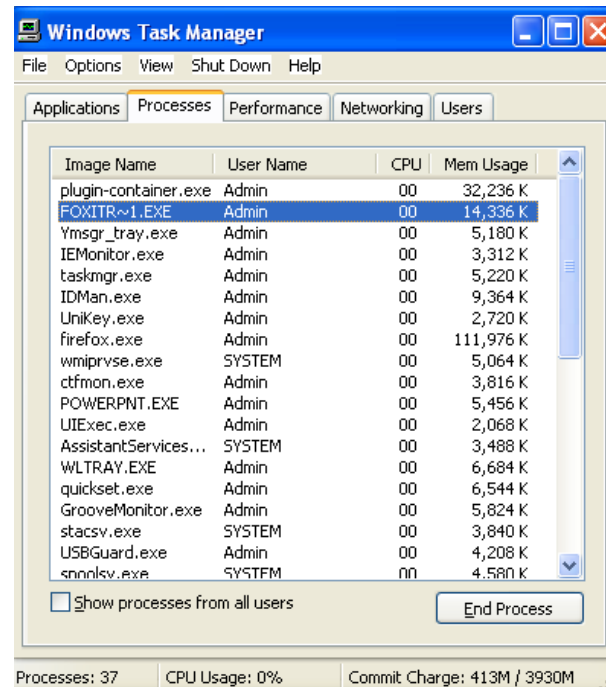
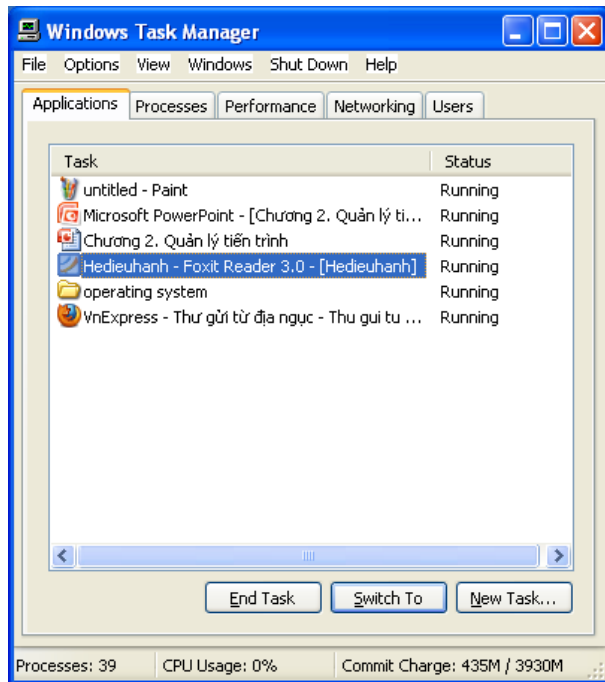
- 1.1. Khái niệm tiến trình
- 1.2. Điều phối tiến trình
- 1.3. Thao tác trên tiến trình
- 1.4. Phân loại tiến trình
- 1.5. Liên lạc giữa các tiến trình

1.1. Khái niệm tiến trình

- Chương trình (program) là một thực thể thụ động chứa đựng các chỉ thị điều khiển máy tính (lệnh) thi hành một tác vụ nào đó. Chương trình được cất trên đĩa dưới dạng file.
- Khi chương trình được nạp vào RAM và CPU bắt đầu thi hành chương trình thì lúc này chương trình trở thành tiến trình (process).

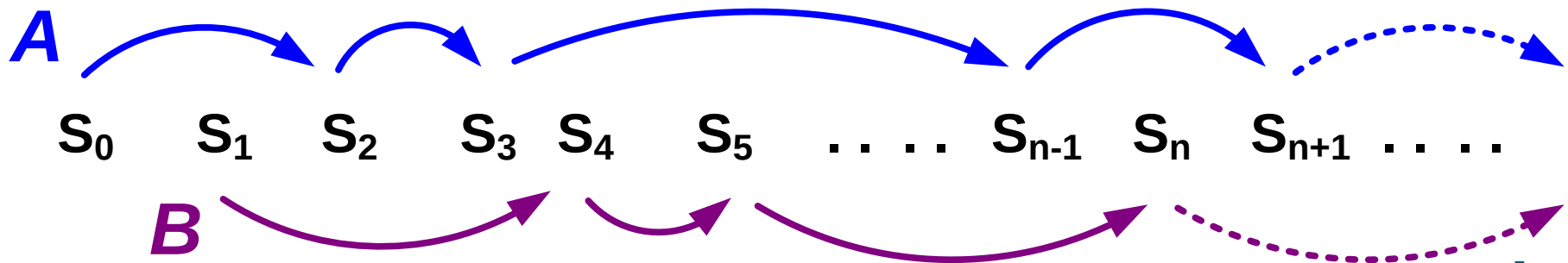
1.1. Khái niệm tiến trình

- Ví dụ: Task manager



1.1. Khái niệm tiến trình

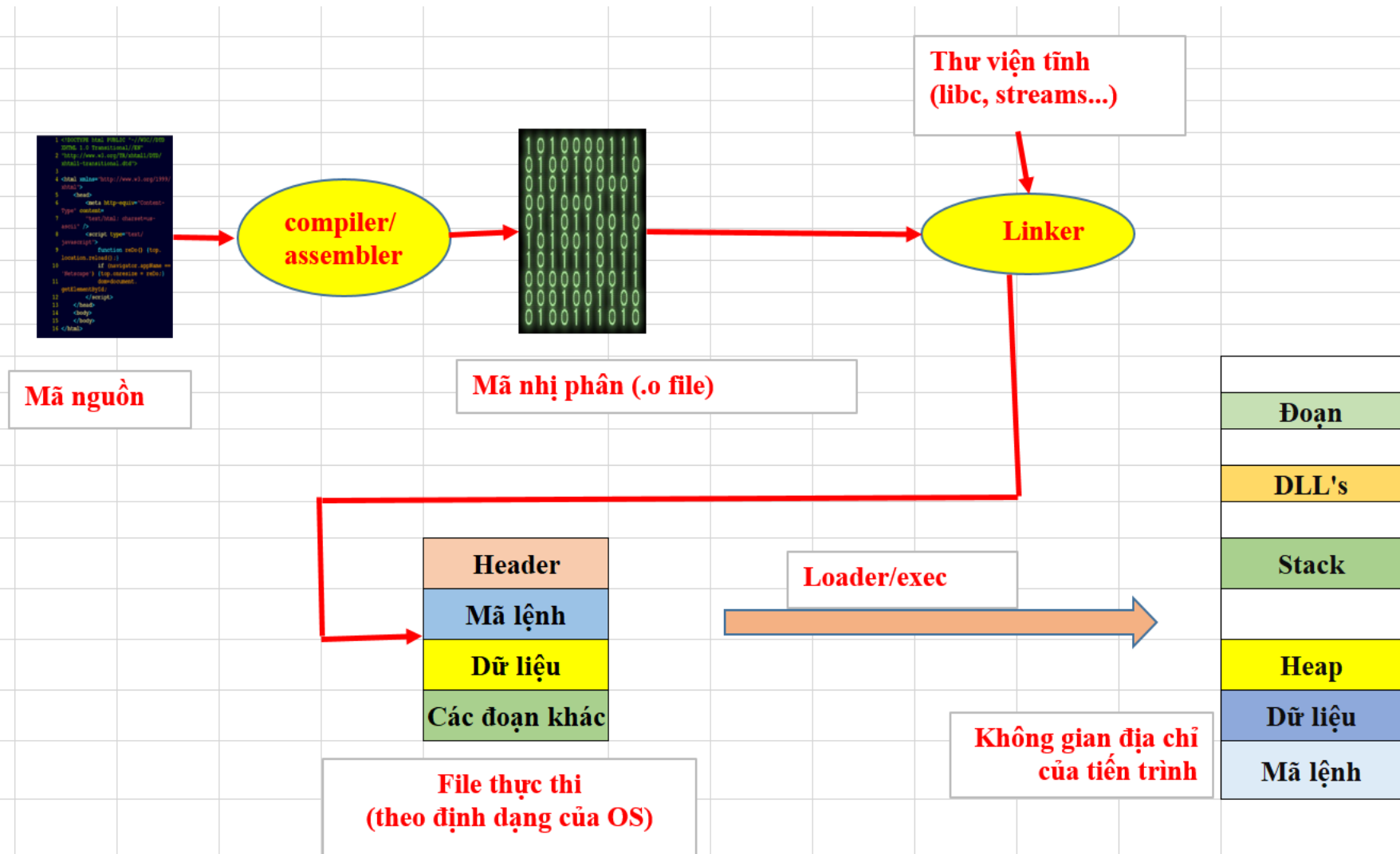
- Là một **dãy thay đổi trạng thái của hệ thống**
 - Chuyển từ trạng thái này sang trạng thái khác được thực hiện theo yêu cầu nằm trong chương trình của người sử dụng
 - Xuất phát từ một trạng thái ban đầu
 - Trạng thái hệ thống:
 - Vi xử lý: Giá trị các thanh ghi
 - Bộ nhớ: Nội dung các ô nhớ
 - Thiết bị ngoại vi: Trạng thái thiết bị



1.1. Khái niệm tiến trình

- Một chương trình có thể:
 - *Được thực thi bởi các tiến trình khác nhau, với các bộ dữ liệu khác nhau*
 - *Gọi tới nhiều tiến trình*
- Tiến trình là một thực thể chủ động: *bộ đếm lệnh, tập tài nguyên*

DỊCH VÀ THỰC THI MỘT CHƯƠNG TRÌNH



DỊCH VÀ THỰC THI MỘT CHƯƠNG TRÌNH

Hệ điều hành tạo một tiến trình và phân phối vùng nhớ cho nó

Bộ thực hiện (loader/exec)

- Đọc và dịch (interprets) file thực thi (header file)
- Thiết lập không gian địa chỉ cho tiến trình để chứa mã lệnh và dữ liệu từ file thực thi
- Đặt các tham số dòng lệnh, biến môi trường (argc, argv, envp) vào stack
- Thiết lập các thanh ghi của VXL tới các giá trị thích hợp và gọi hàm “_start()” (hàm của hệ điều hành)

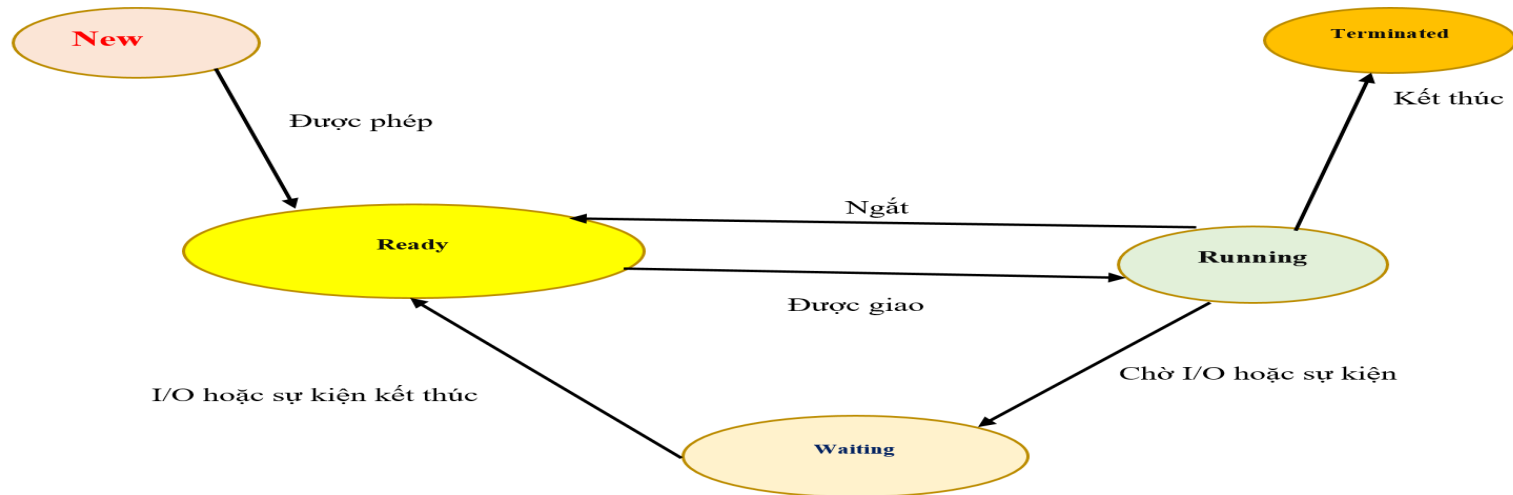
Chương trình bắt đầu thực hiện tại “_start()”. Hàm này gọi tới hàm main() (hàm của chương trình)

-> “Tiến trình” đang thực hiện, không còn đề cập đến “chương trình” nữa

Khi hàm main() kết thúc, OS gọi tới hàm “_exit()” để hủy bỏ tiến trình và thu hồi tài nguyên

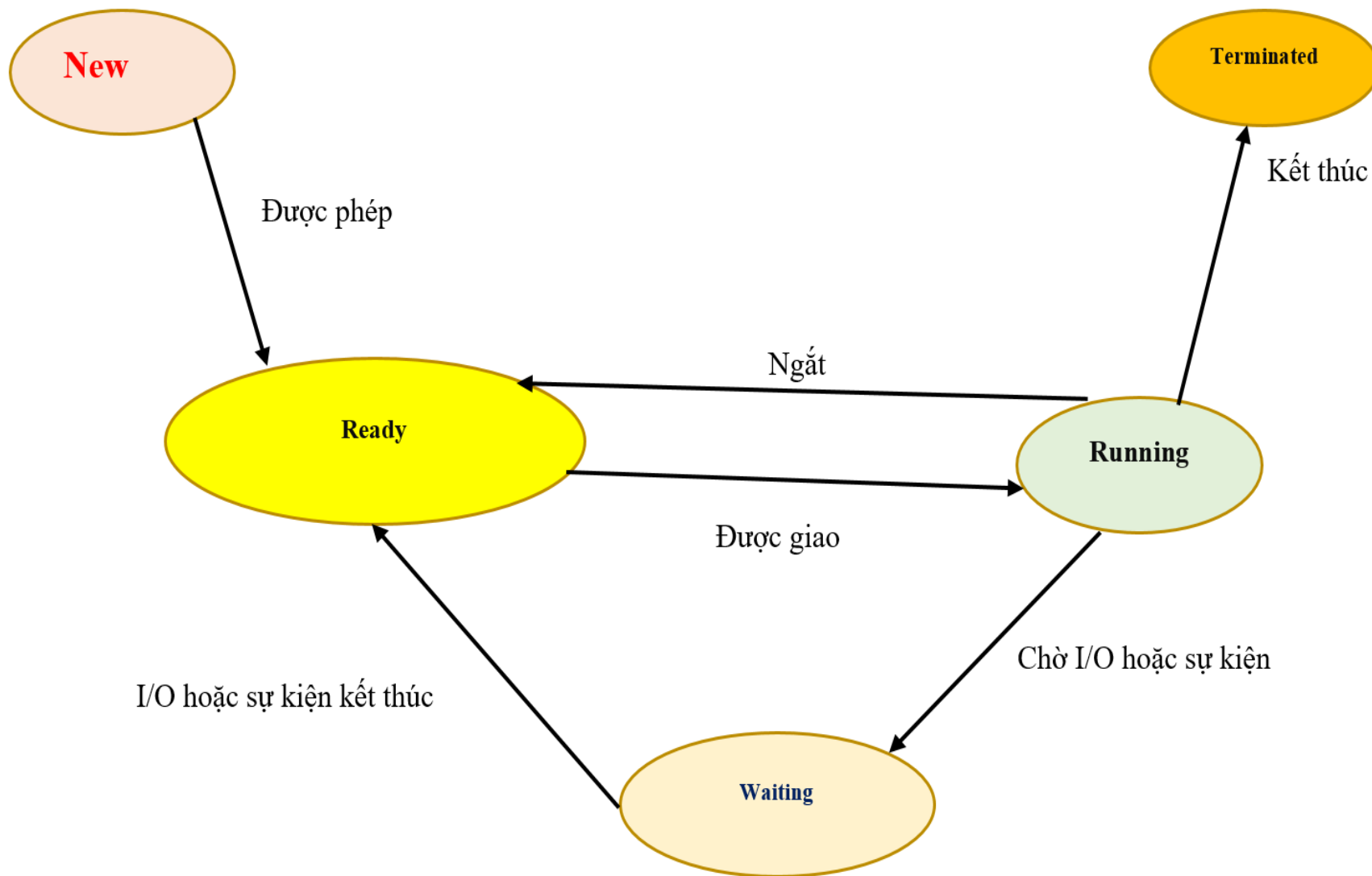
1.1. Khái niệm tiến trình

- *Các trạng thái của một tiến trình*



- New* : Tiến trình được tạo lập
- Ready* : Tiến trình chờ cấp phát CPU để xử lý
- Running* : Tiến trình đang được xử lý
- Waiting* : Tiến trình dừng vì thiếu tài nguyên hay chờ sự kiện nào đó
- Terminated* : Tiến trình hoàn thành

- **Chú ý:** Tại 1 thời điểm chỉ có 1 TT ở trạng thái **Running** với 1 bộ VXL. Trong khi có thể có **nhiều TT** ở trạng thái Ready hay Waiting



Các cung chuyển tiếp trong sơ đồ trạng thái biểu diễn sáu sự chuyển trạng thái có thể xảy ra trong các điều kiện sau:

- **Được phép:** Tiến trình mới tạo, nếu bộ nhớ còn trống, sẽ được đưa vào bộ nhớ và sẵn sàng nhận CPU, khi đó tiến trình từ trạng thái **New** được chuyển sang trạng thái **Ready**
- **Được giao:** Bộ điều phối cấp phát cho tiến trình một khoảng thời gian sử dụng CPU và cho tiến trình thực hiện, khi đó tiến trình từ trạng thái **Ready** được chuyển sang trạng thái **Running**
- **Kết thúc:** Khi tiến trình kết thúc việc thực hiện, khi đó tiến trình từ trạng thái **Running** được chuyển sang trạng thái **End**
- **Chờ I/O hoặc sự kiện:** Khi tiến trình yêu cầu một tài nguyên nhưng chưa được đáp ứng vì tài nguyên chưa sẵn sàng hoặc tiến trình chờ thao tác nhập/xuất hoàn thành hoặc tiến trình chờ một sự kiện nào đó, khi đó tiến trình được chuyển từ trạng thái **Running** sang trạng thái **Waiting**
- **Ngắt:** Khi tiến trình tạm dừng vì hết thời gian sử dụng CPU, bộ điều phối sẽ chọn một tiến trình khác để xử lý, khi đó tiến trình được chuyển từ trạng thái **Running** sang trạng thái **Ready**
- **I/O hoặc sự kết thúc:** Khi tài nguyên mà tiến trình yêu cầu trở nên sẵn sàng để cấp phát, hay sự kiện hoặc thao tác nhập/ xuất mà tiến trình đang đợi hoàn tất, khi đó bộ tiến trình được chuyển từ trạng thái **Waiting** sang trạng thái **Ready**

Cấu trúc của khối quản lý tiến trình (PCB) Process Control Block

Các con trỏ (địa chỉ liên kết)	Trạng thái tiến trình
Số hiệu tiến trình	
Bộ đếm lệnh	
Nội dung các thanh ghi của CPU	
Thông tin quản lý bộ nhớ	
Danh sách các file đang mở	
Con trỏ tới PCB khác Thông tin thống kê Thông tin tài nguyên có thể sử dụng Thông tin dùng để điều phối tiến trình	

Cấu trúc của khối quản lý tiến trình (PCB)

Process Control Block

Mỗi tiến trình được thể hiện trong hệ thống bởi một khối điều khiển tiến trình PCB chứa các thông tin với mỗi tiến trình gồm có: để xác định duy nhất một TT

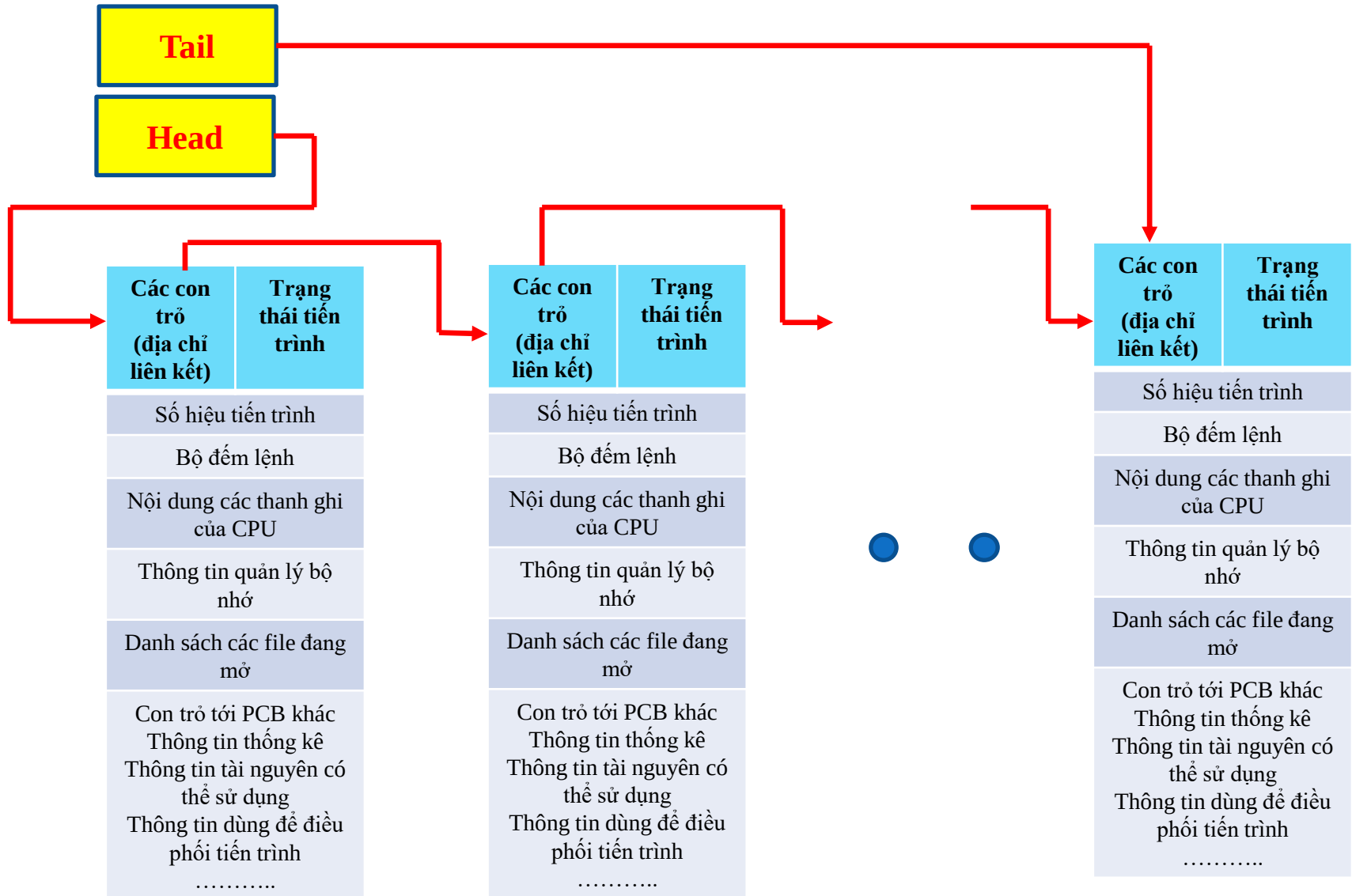
- Process ID, Parent process ID
- Trạng thái tiến trình: new, ready, running, waiting, end
- Program counter: Địa chỉ của lệnh kế tiếp sẽ thực thi
- Các thanh ghi CPU
- Thông tin dùng để định thời CPU : priority...
- Thông tin bộ nhớ : base/limit register, page tables
- Thông tin thống kê: CPU time, time limits
- Thông tin trạng thái I/O: danh sách thiết bị I/O được cấp phát, danh sách các file đang mở..
- Con trỏ (pointer) đến PCBs khác

PCB đơn giản phụ vụ như kho chứa cho bất cứ thông tin khác nhau từ quá trình này tới quá trình khác

Mục tiêu của PCB

- Bảo đảm một số lượng hợp lệ các tiến trình truy xuất đồng thời đến các tài nguyên không thể chia sẻ được
- Cấp phát tài nguyên cho tiến trình có yêu cầu trong một khoảng thời gian trì hoãn có thể chấp nhận được
- Tối ưu hóa sự sử dụng tài nguyên

Danh sách các tiến trình



1.1. Khái niệm tiến trình

- **Tiến trình đơn luồng**: Là tiến trình thực hiện chỉ 1 luồng thực thi

Có một luồng câu lệnh thực thi

-> Cho phép thực hiện chỉ một nhiệm vụ tại một thời điểm

- **Tiến trình đa luồng** : Là tiến trình có nhiều luồng thực thi -> cho phép thực hiện nhiều hơn một nhiệm vụ tại một thời điểm

1.2. Điều phối tiến trình

Mục đích Sử dụng tối đa thời gian của CPU

-> Cần có nhiều tiến trình trong hệ thống

Vấn đề Luân chuyển CPU giữa các tiến trình

-> Phải có hàng đợi cho các tiến trình

Hệ thống một processor

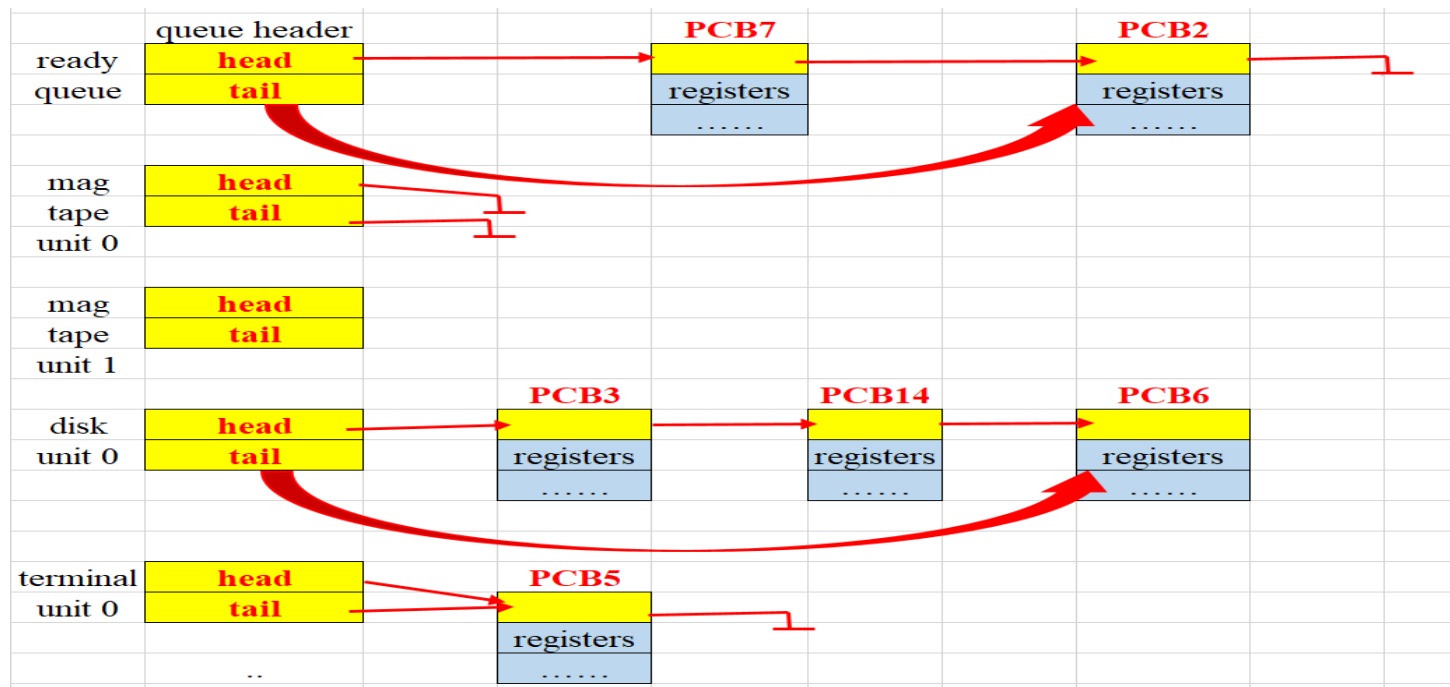
-> Một tiến trình thực hiện

-> Các tiến trình khác phải đợi tới khi CPU tự do

Các danh sách điều phối

Hệ thống có nhiều hàng đợi dành cho tiến trình

- **Job-queue**: Tập các tiến trình trong hệ thống
- **Ready – queue**: Tập các tiến trình tồn tại trong bộ nhớ, đang sẵn sàng và chờ đợi để được thực hiện
- **Device queues** tập các **tiến** trình đang chờ đợi một thiết bị vào ra. Phân biệt hàng đợi cho từng thiết bị



Các danh sách điều phối

- Mỗi tài nguyên có một hàng đợi lưu danh sách các tiến trình đang đợi tài nguyên
- Khi một Tiến trình được tạo, PCB của tiến trình sẽ chèn vào danh sách tác vụ (job list). Khi bộ nhớ đủ chỗ, một tiến trình trong danh sách tác vụ được chọn, nạp từ đĩa vào bộ nhớ và PCB của tiến trình đó được chuyển sang danh sách sẵn sàng (ready list). Bộ điều phối sẽ chọn một tiến trình trong danh sách sẵn sàng và cấp CPU cho tiến trình đó. Tiến trình được cấp CPU sẽ thi hành và sẽ chuyển sang danh sách chờ đợi (waiting list) khi xảy ra các sự kiện, I/O
- Tiến trình đang thi hành có thể bị bắt buộc tạm dừng xử lý do một ngắt xảy ra, khi đó tiến trình được đưa trở lại vào danh sách sẵn sàng để chờ được cấp CPU cho lượt tiếp theo.
- HĐH chỉ sử dụng một danh sách tác vụ, một danh sách sẵn sàng nhưng mỗi một tài nguyên (thiết bị ngoại vi) có một danh sách chờ đợi riêng bao gồm các tiến trình đang chờ được cấp phát tài nguyên đó

Khi có yêu cầu tài nguyên mà chưa được đáp ứng, tiến trình được đưa vào hàng đợi tài nguyên

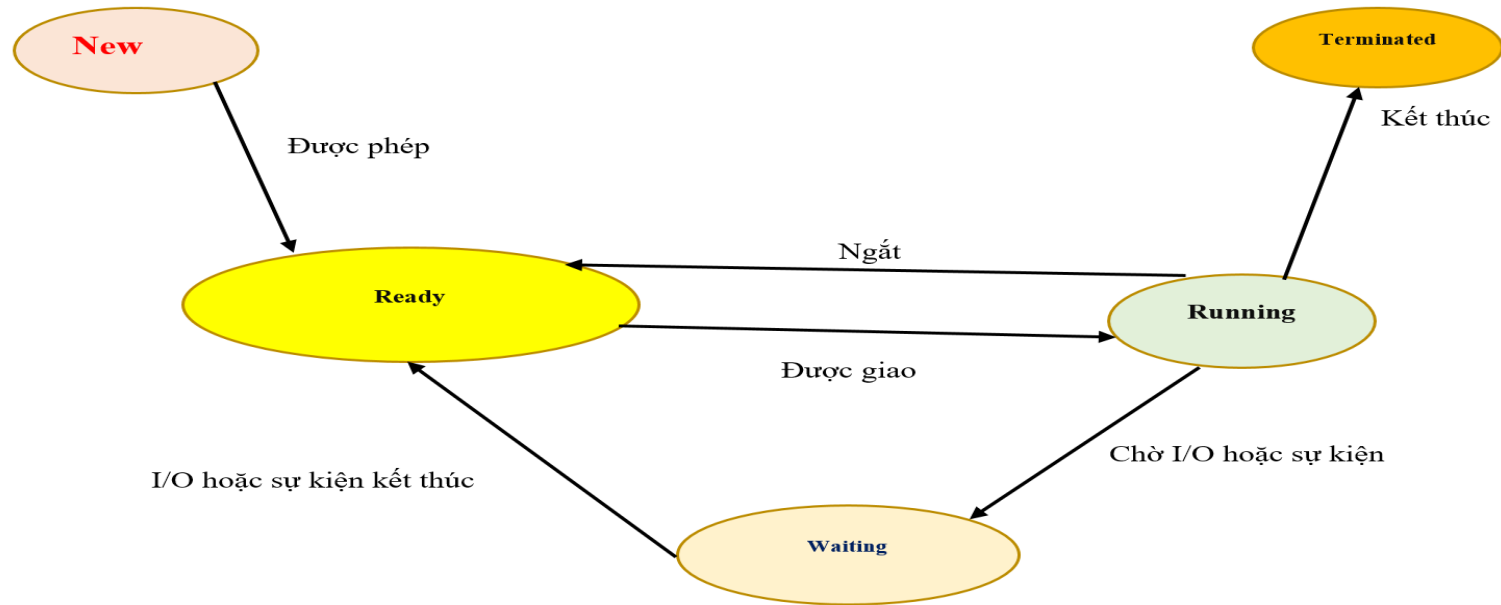


- Các tiến trình di chuyển giữa hàng đợi khác nhau
- Tiến trình mới tạo, được đặt trong hàng đợi sẵn sàng, và đợi cho tới khi được lựa chọn để thực hiện

Các hàng đợi tiến trình

- Tiến trình đã được chọn và đang thực hiện
 1. Đưa ra một yêu cầu vào ra: đợi trong một hàng đợi thiết bị
 2. Tạo một tiến trình con và đợi tiến trình con kết thúc
 3. Hết thời gian sử dụng CPU, phải quay lại hàng đợi sẵn sàng
- Trường hợp (1+ 2) sau khi sự kiện chờ đợi hoàn thành
 - Tiến trình sẽ chuyển từ trạng thái đợi sang trạng thái sẵn sàng
 - Tiến trình quay lại hàng đợi sẵn sàng
- Tiến trình tiếp tục chu kỳ (sẵn sàng, thực hiện, chờ đợi) cho tới khi kết thúc
 - Xóa khỏi tất cả các hàng đợi
 - PCB và tài nguyên đã cấp được giải phóng

Bộ điều phối



Lựa chọn tiến trình trong các hàng đợi

- Điều phối công việc (Job scheduler; Long – term scheduler)
- Điều phối CPU (CPU scheduler; short-term scheduler)
- Swap tiến trình (Medium – term scheduler)

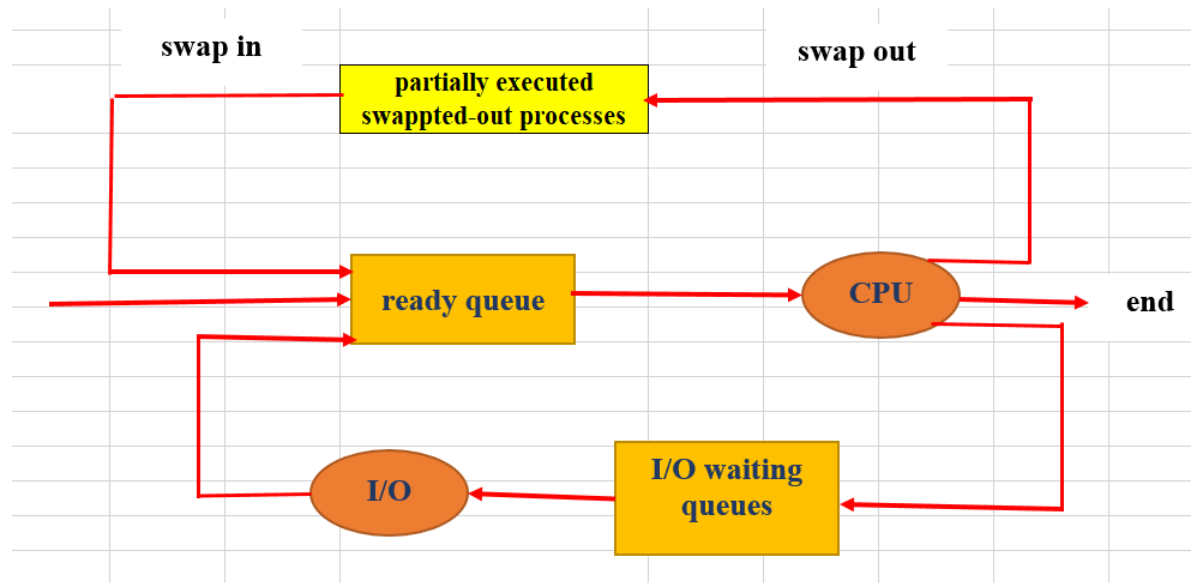
Điều phối công việc

- Chọn các tiến trình từ hàng đợi tiến trình được lưu trong các vùng đệm (đĩa từ) và đưa vào bộ nhớ để thực hiện
- Thực hiện không thường xuyên (đơn vị giây / phút)
- Điều khiển mức độ đa chương trình (số tiến trình trong bộ nhớ)
- Khi mức độ đa chương trình ổn định, điều phối công việc được gọi chỉ khi có tiến trình rời khỏi hệ thống
- Vấn đề lựa chọn công việc
 - ✓ Tiến trình thiên về vào/ra: sử dụng ít thời gian CPU
 - ✓ Tiến trình thiên về tính toán: sử dụng nhiều thời gian CPU
 - ✓ Cần lựa chọn lẫn cả 2 loại tiến trình
 - > TT vào ra: hàng đợi sẵn sàng rỗng, lãng phí CPU
 - > TT tính toán: hàng đợi thiết bị rỗng, lãng phí thiết bị

Điều phối CPU

- Lựa chọn một tiến trình từ hàng đợi các tiến trình đang sẵn sàng thực hiện và phân phối CPU cho nó
- Được thực hiện thường xuyên (VD: 100ms/lần)
 - Tiến trình thực hiện vài ms rồi thực hiện vào ra
 - Lựa chọn tiến trình mới, đang sẵn sàng
- Phải thực hiện nhanh
 - 10ms để quyết định $\rightarrow 10/(110)=9\%$ thời gian CPU lãng phí
- Vấn đề luân chuyển CPU từ tiến trình này tới tiến trình khác
 - Phải lưu trạng thái của tiến trình cũ (PCB) và khôi phục trạng thái cho tiến trình mới
 - Thời gian luân chuyển là lãng phí
 - Có thể được hỗ trợ bởi phần cứng
- Vấn đề lựa chọn tiến trình (điều phối CPU)

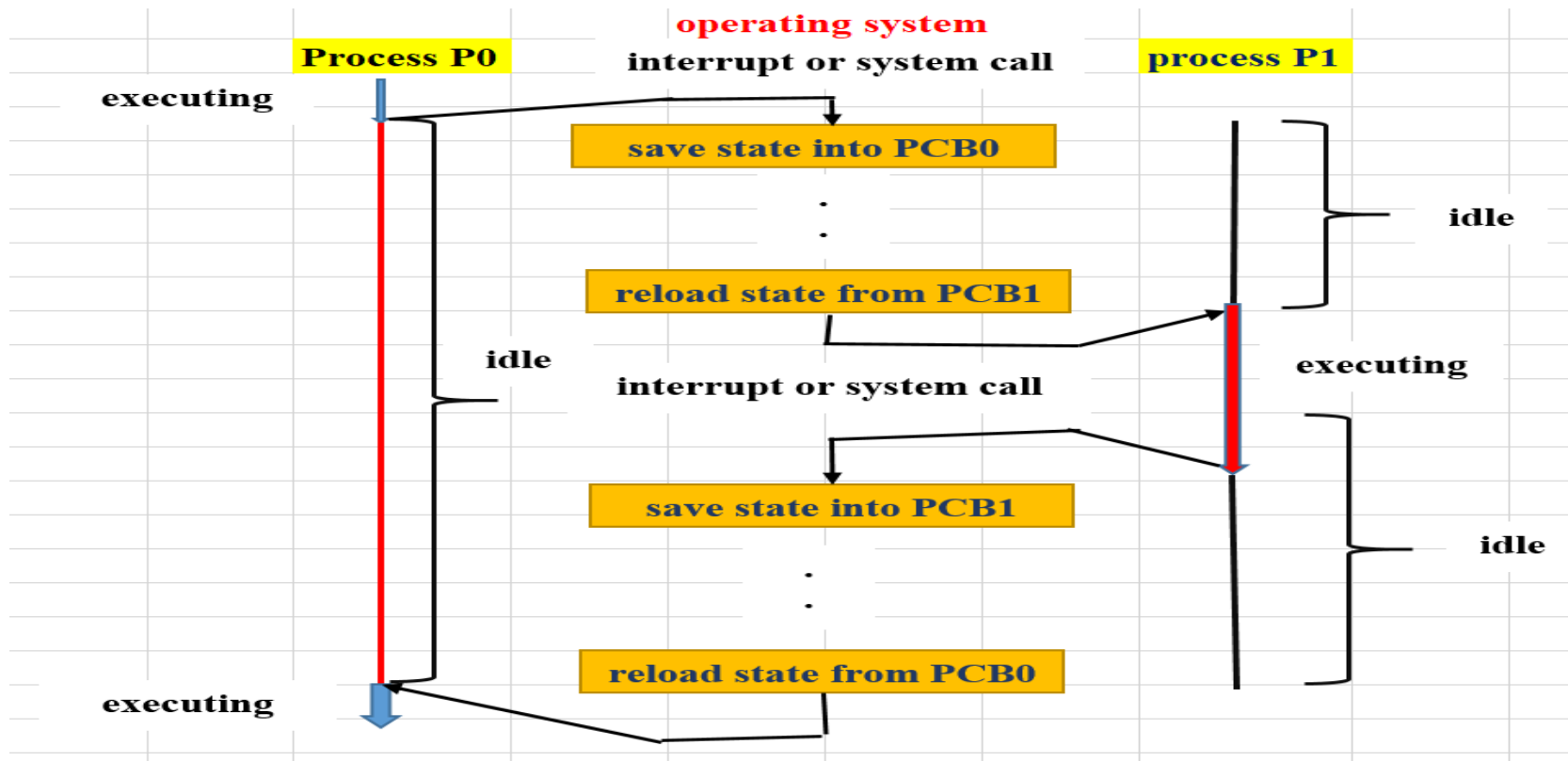
Swapping tiến trình (Medium-term scheduler)



- **Nhiệm vụ**
 - Đưa tiến trình ra khỏi bộ nhớ (làm giảm mức độ đa chương trình)
 - Sau đó đưa tiến trình quay lại (có thể ở vị trí khác) và tiếp tục thực hiện
- **Mục đích:** Giải phóng vùng nhớ, tạo vùng nhớ tự do rộng hơn

Chuyển ngữ cảnh (context switch)

- Chuyển CPU từ tiến trình này sang tiến trình khác (hoán đổi tiến trình thực hiện)
- Thực hiện khi xuất hiện tín hiệu ngắt (ngắt thời gian) hoặc tiến trình đưa ra lời gọi hệ thống (thực hiện vào ra)
- Lưu đồ của chuyển CPU giữa các tiến trình



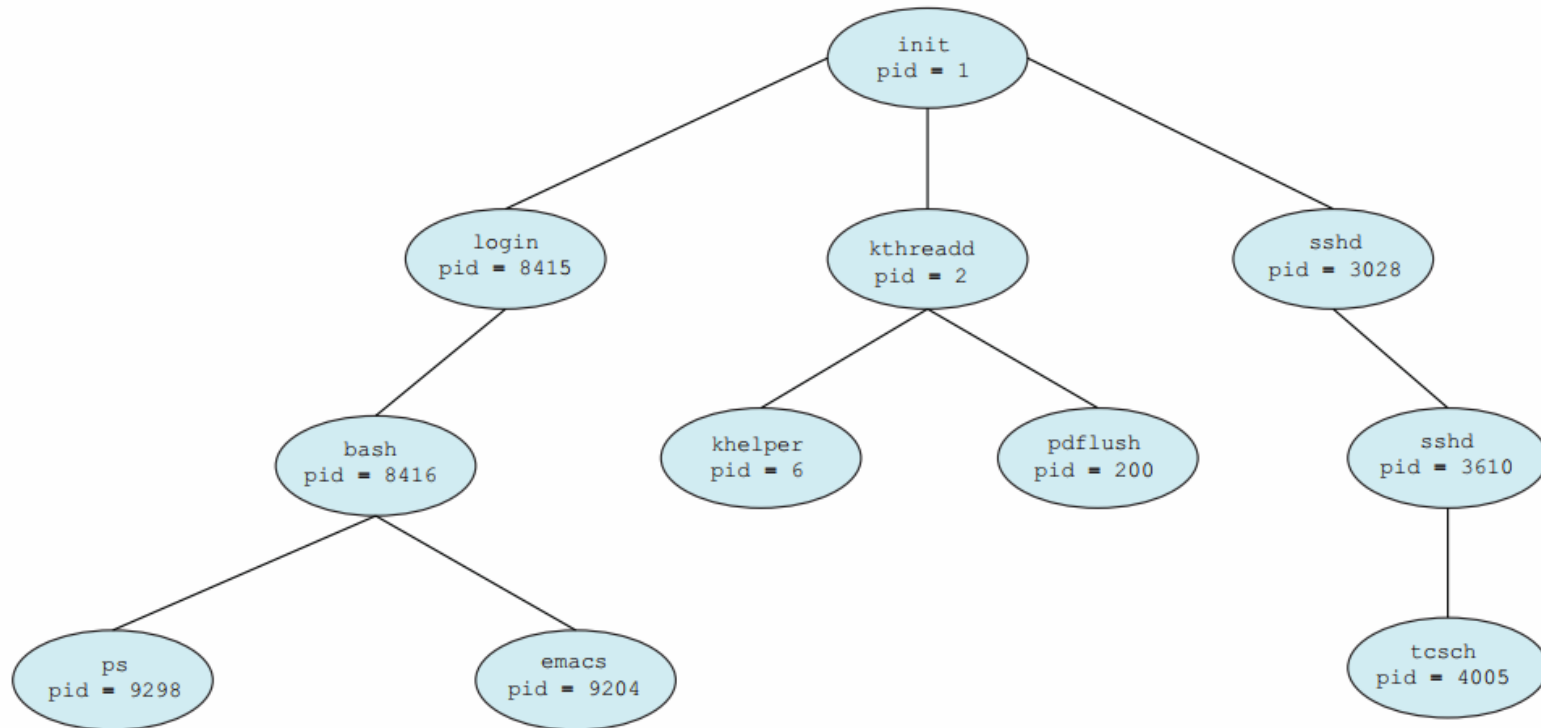
1.3. Thao tác trên tiến trình

- Tạo tiến trình
- Kết thúc tiến trình

Tạo tiến trình

- Tiến trình có thể tạo nhiều tiến trình mới cùng hoạt động (`CreateProcess()`, `fork()`)
 - Tiến trình tạo: tiến trình cha
 - Tiến trình được tạo: tiến trình con
- Tiến trình con có thể tạo tiến trình con khác -> cây tiến trình
- Vấn đề phân phối tài nguyên
 - Tiến trình con lấy tài nguyên từ hệ điều hành
 - Tiến trình con lấy tài nguyên từ tiến trình cha
 - Tất cả các tài nguyên
 - Một phần tài nguyên của tiến trình cha (ngăn ngừa việc tạo quá nhiều tiến trình con)
- Vấn đề thực hiện
 - Tiến trình cha tiếp tục thực hiện đồng thời với tiến trình con
 - Tiến trình cha đợi tiến trình con kết thúc

Tạo tiến trình



Kết thúc tiến trình

- Hoàn thành câu lệnh cuối và yêu cầu HĐH xóa nó (exit)
 - Gửi trả dữ liệu tới tiến trình cha
 - Các tài nguyên đã cung cấp được trả lại hệ thống
- Tiến trình cha có thể kết thúc sự thực hiện của tiến trình con
 - Tiến trình cha phải biết định danh tiến trình con -> tiến trình con phải gửi định danh cho tiến trình cha khi được khởi tạo
 - Sử dụng lời gọi hệ thống (abort)
- Tiến trình cha kết thúc tiến trình con khi
 - Tiến trình con sử dụng vượt quá tài nguyên được cấp
 - Nhiệm vụ cung cấp cho tiến trình con không còn cần thiết nữa
 - Tiến trình cha kết thúc và hệ điều hành không cho phép tiến trình con tồn tại khi tiến trình cha kết thúc -> Cascading termination . VD, kết thúc hệ thống